Towards protocol formal analysis

- Need mathematical guarantees of protocol correctness
- Formal analysis can provide some
- A number of approaches exist to formal protocol analysis
  - Logics of authentication
  - Process calculi
  - Ad-hoc languages
  - Mathematical induction
  - …
- Best results when support of computer tools is available
  - Dealing with large protocols
  - Automatic findings about protocols
    - Finding attacks?
    - Claiming a protocol correct, i.e. it achieves its goals?

Main computer tools to assist with protocol analysis

- Model checking (state enumeration)
  - Aimed at searching attacks; in brief:
    - Specify target system as a set of states
    - Each state can be thought of as a photograph of the running system
    - Build finite state machine, i.e. graph of reachable states
    - Check all states for attacks, i.e. for negated property
  - Offers support to various protocol analysis approaches, notably process calculi
- Theorem proving
  - Aimed at proving correctness; in brief:
    - Specify target system by induction, e.g. 0 is a number if n is a number then so is suc(n)
    - Prove properties of specified system again by induction, examples later
  - Mainly offers support to mathematical induction or ad hoc languages

Example of finite state machine (portion)

- State 1:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$
  - B knows B, K_b, K_{b-1}, A, K_A
- State 2:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$
  - B knows B, K_b, K_{b-1}, A, K_A, $\{A,N_a\}K_b$, N_b,$\{N_a,N_b\}K_a$
- State 3:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$,$\{N_a,N_b\}K_a$, N_b
  - B knows B, K_b, K_{b-1}, A, K_A, $\{A,N_a\}K_b$, N_b,$\{N_a,N_b\}K_a$
- …

- Note encoding of message sending and receiving
- Which protocol is it?
- How many states must be built?

Example of finite state machine (portion)

- State 1:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$
  - B knows B, K_b, K_{b-1}, A, K_A
- State 2:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$
  - B knows B, K_b, K_{b-1}, A, K_A, $\{A,N_a\}K_b$, N_a, N_b,$\{N_a,N_b\}K_a$
- State 3:
  - A knows A, K_a, K_{a-1}, B, K_b, N_a, $\{A,N_a\}K_b$,$\{N_a,N_b\}K_a$, N_b
  - B knows B, K_b, K_{b-1}, A, K_A, $\{A,N_a\}K_b$, N_a, N_b,$\{N_a,N_b\}K_a$
- …

- Consider target property: B ignores N_b
  - Where is it violated?
  - Is it an attack?
- State explosion risk

Initial evaluation

- Pros
  - Both liveness and safety properties
  - Analysis fully automatic
  - Mild learning curve, hence many industrial applications
- Cons
  - Bounded types
  - Bounded protocol session interleaving
  - Specification of limitation of DY^+

Example of inductive reasoning (reminder!)

- Inductive specification of natural numbers
  - Base case: 0 is natural
  - Inductive case: if n is natural then so is suc(n)
- Inductive proof of a property about the natural numbers
  - The sum S_n of the first n naturals is n(n+1)/2
- Because target system specified by induction, induction can also be used as proof principle (technique) as follows
  - Base proof case: thesis clearly holds of base case
  - Inductive proof case: let thesis hold for n; aim at proving

    $S_{n+1} = (n+1)(n+2)/2$

    We have $(n+1)(n+2)/2 = n^2+3n+2 = (n+1)+(n(n+1)/2) = n+1+S_n = S_{n+1}$
    QED

The Inductive Method

- A method of formal protocol analysis
- Based on formal language Higher Order Logics (HOL)
  - Propositional Logic: no quantification
  - FOL: quantification only over variables
  - HOL: quantification also over funtional or predicate symbols

    $\forall P\ x.\ P(x) \vee \neg P(x)$

    $\forall x\ y.\ x=y \leftrightarrow \forall P.\ (P(x) \leftrightarrow P(y))$
- So, HOL is the language to specify a protocol and its goals
- Induction used as a principle
  - For the protocol specification
  - For the proofs
- Proof is the counterpart of state enumeration for MC
  - It means to check that a specification meets a property/goal or not

Initial evaluation (almost opposite to MC)

- Pros
  - Unbounded types
  - Unbounded protocol session interleaving
  - Specification of full DY$^+$
- Cons
  - No liveness properties, only safety, as we shall see
  - Analysis not fully automatic, rather, interactive
  - Steep learning curve

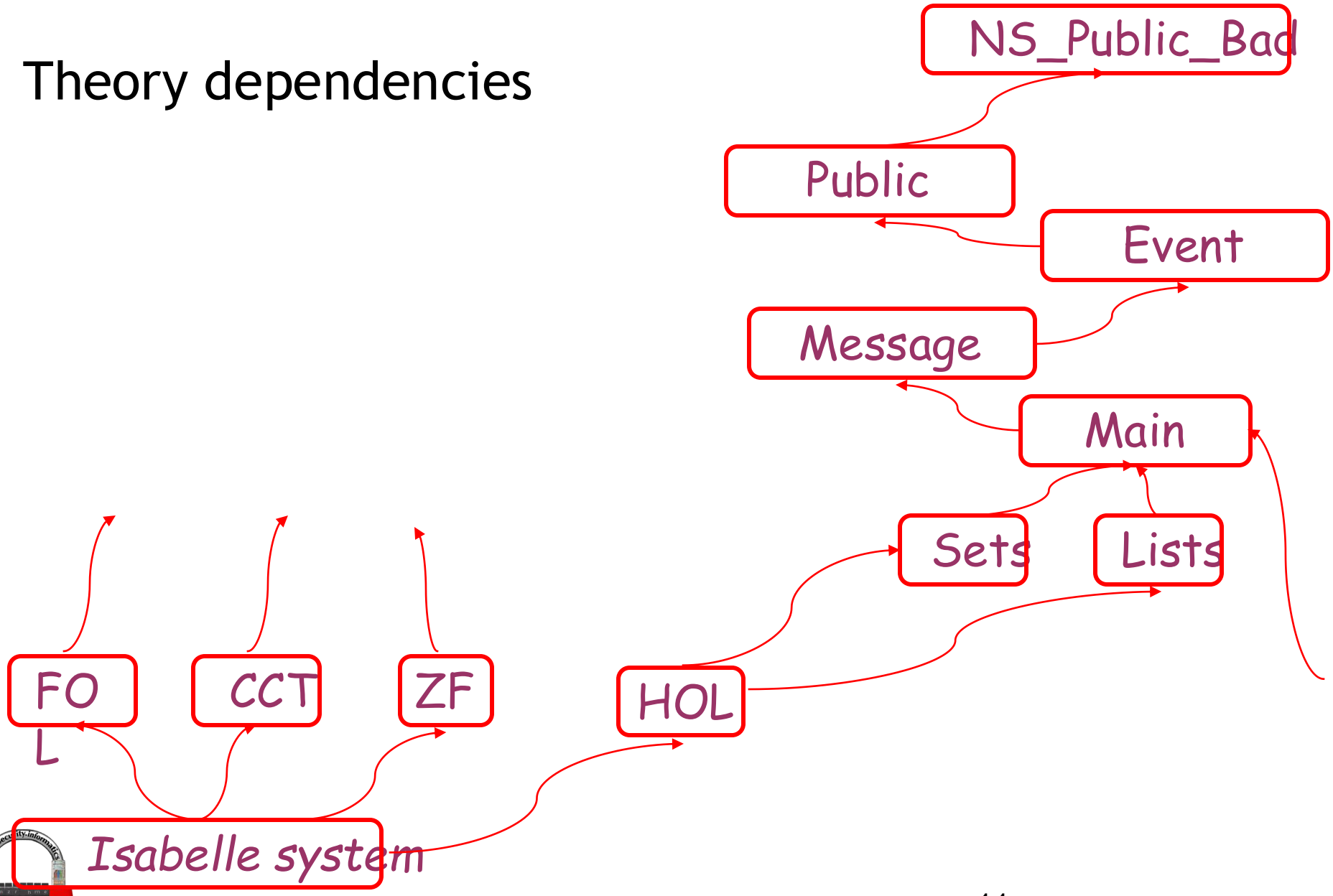
Isabelle (Cambridge and Munich)

- Inductive Method mechanised (runs in) Isabelle
- Isabelle is a theorem prover capable of
  - Conditional simplification (term-rewriting)
    E.g. X = (If 2=1+3 then Y else Z) gets rewritten as X=Z
  - Classical reasoning
    E.g. Answer yes to $P(X) \wedge Q(X) \rightarrow P(X)$
- Main methods (used to be tactics)
  - simp launches the simplifier
  - blast launches the classical reasoner
  - force combines them
  - auto similar to force but applies to entire proof state, as we shall see
  - clarify does obvious steps, such as $P(X) \wedge Q(X)$ means both holds
- Can be downloaded and used for free

Using the Inductive Method in practice

- A theory file is a file containing a specification of the target system and the proof scripts of the proven theorems
- Download Isabelle and install it
  - Straightforward in Linux, needs virtual machine on Windows
  - It comes with
    - ML, a functional language, so computation is by rewriting
    - ProofGeneral, the graphical interface
    - A large repository of existing work, separated in directories containing theory files
      - In particular, security protocol theories are under /src/HOL/Auth
    - A precompiled database of HOL theories
- Launch Isabelle on HOL database of theories
  - Use theories Message.thy then Event.thy then Public.thy then a chosen protocol theory

Theory dependencies



Features of the protocol model

- Putting all concepts together
  - Protocol model is a formal specification of the real protocol in HOL by means of induction within Isabelle
- Protocol model provides operational semantics to real prot
  - A basic event is to send a message or to receive it (one more exists)
  - A trace is one possible interleaving of events, possibly belonging to different protocol sessions, and including attacker's events
  - The protocol model is the set of all possible traces using the protocol in presence of a DY^{+} attacker
- Protocol model defined (or, specification of the protocol given) by structural induction on the length of traces

Example

- Please only focus on structure of the 5 rules for now!!
  - One base case (obviously!) plus 4 inductive steps

```
Nil:  "[] ∈ flp"

Fake: "⟦evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))⟧
       ⟹ Says Spy B X # evsf ∈ flp"

FLP1: "⟦evs1 ∈ flp; Nonce Na ∉ used evs1⟧
       ⟹ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "⟦evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2⟧
       ⟹ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "⟦evsr ∈ flp; Says A B X ∈ set evsr⟧
       ⟹ Gets B X # evsr ∈ flp"
```

Main types: cryptographic keys

- Key type is unbounded
- Function to invert a key, idempotent
- Symmetric keys remain the same when inverted

```
types key = nat
consts invKey          :: "key=>key"

specification (invKey)
  invKey [simp]: "invKey (invKey K) = K"

definition symKeys :: "key set" where
  "symKeys == {K. invKey K = K}"
```

Main types: agents

- One TTP
- Unbounded number of protocol executors
- One attacker, Spy
- Datatype specifies the only type allowed type constructors
  - 3 in this case

```
datatype agent = Server | Friend nat | Spy
```

Main types: messages

- 7 type constructors
- Concatenated messages can be written nicely

```
datatype msg = Agent  agent      --{*Agent names*}
         | Number nat          --{*Integers, timestamps, …*}
         | Nonce  nat          --{*Unguessable nonces*}
         | Key    key          --{*Crypto keys*}
         | Hash   msg          --{*Hashing*}
         | MPair  msg msg      --{*Concatenation*}
         | Crypt  key msg      --{*Ciphertext*}

translations
  "{|x, y, z|}"   == "{|x, {|y, z|}|}"
  "{|x, y|}"      == "CONST MPair x y"
```

Main types: events

- 3 main ones but others can be defined for specific protocols

$$
\begin{array}{l}
\textbf{datatype} \\
\quad event = Says \;\; agent \;\; agent \;\; msg \\
\qquad\quad | \;\; Gets \;\; agent \qquad\;\; msg \\
\qquad\quad | \;\; Notes \;\; agent \qquad\;\; msg
\end{array}
$$

An example trace

- Traces typically built from right to left, here bottom to top
- Built randomly
- It's not obvious to which protocol model this trace belongs…

```
Says Spy D (Nonce Nc)
Notes Spy (Nonce Nc)
Notes Spy (Nonce Nc)
Says B C (Crypt (priSK B)(Nonce Nc))
Notes B (Nonce Nc)
Says Spy D (Nonce Na')
Says C B {|Agent C, Nonce Nc|}
Notes Spy (Nonce Na')
Says A B {|Agent A, Nonce Na'|}
Says A B {|Agent A, Nonce Na|}
```

Encoding the protocol steps as inductive definition

- Each protocol step mapped to inductive case, e.g. (semiformal)
- Focus on each rule precondition: Says A' B X or Gets X?

$A \rightarrow B : \{A, N_a\}_{Kb}$

If ev is a trace and Na is unused on it, then extend trace with
Says A B Crypt(pk B){A,Na}

$B \rightarrow A : \{N_a, N_b\}_{Ka}$

If Says A' B Crypt(pk B){A,X} is on a trace and Nb is unused on it, then extend trace with
Says B A Crypt(pk A){X,Nb}

$A \rightarrow B : \{N_a\}_{Kb}$

If Says A B Crypt (pk B){A,Na} and Says B' A Crypt(pk A){Na,X} is on a trace, then extend trace with
Says A B Crypt(pk B){X}

Example

- Here's a protocol and its inductive model
  - Note base case and reception case
  - Fake sets the protocol against DY⁺ attacker… how?

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : sign(B, N_a)$

```
Nil:  "[] ∈ flp"

Fake: "⟦evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))⟧
       ⟹ Says Spy B X # evsf ∈ flp"

FLP1: "⟦evs1 ∈ flp; Nonce Na ∉ used evs1⟧
       ⟹ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "⟦evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2⟧
       ⟹ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "⟦evsr ∈ flp; Says A B X ∈ set evsr⟧
       ⟹ Gets B X # evsr ∈ flp"
```

Main operators: parts

- It extracts all components from a message set as if ciphertexts were broken
  - Hence the attacker cannot apply it!

inductive-set

$parts :: msg\ set => msg\ set$

for $H :: msg\ set$

where

$Inj\ [intro]: \quad X \in H ==> X \in parts\ H$

$| \ Fst: \quad \{|X, Y|\} \in parts\ H ==> X \in parts\ H$

$| \ Snd: \quad \{|X, Y|\} \in parts\ H ==> Y \in parts\ H$

$| \ Body: \quad Crypt\ K\ X \in parts\ H ==> X \in parts\ H$

Main operators: used

- It serves to encode freshness
  - Hence if a nonce is unused on a trace then it is fresh on it
  - Gets case seems weird by think of protocol model…
- initState skipped though obvious

```
consts

  used :: event list => msg set

primrec
  used-Nil:   used []          = (UN B. parts (initState B))
  used-Cons:  used (ev # evs) =
                    (case ev of
                     Says A B X => parts {X} ∪ used evs
                     | Gets A X  => used evs
                     | Notes A X  => parts {X} ∪ used evs)
```

Example

- Use of fresh nonces now clear!
  - Freshness refers to given trace

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : sign(B, N_a)$

```
Nil:  "[] ∈ flp"

Fake: "⟦evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))⟧
       ⟹ Says Spy B X # evsf ∈ flp"

FLP1: "⟦evs1 ∈ flp; Nonce Na ∉ used evs1⟧
       ⟹ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "⟦evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2⟧
       ⟹ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "⟦evsr ∈ flp; Says A B X ∈ set evsr⟧
       ⟹ Gets B X # evsr ∈ flp"
```

Main operators: analz

- More realistically than parts, it opens ciphertexts coherently
  - Hence it makes sense to have the attacker apply it

inductive-set
$analz :: msg\ set => msg\ set$
for $H :: msg\ set$
where
$Inj\ [intro,simp]: \quad X \in H ==> X \in analz\ H$
$| \ Fst: \quad \{|X,Y|\} \in analz\ H ==> X \in analz\ H$
$| \ Snd: \quad \{|X,Y|\} \in analz\ H ==> Y \in analz\ H$
$| \ Decrypt\ [dest]:$
$[|Crypt\ K\ X \in analz\ H;\ Key(invKey\ K): analz\ H|] ==> X \in analz\ H$

Main operators: synth

- It forms messages by concatenation and encryption from available components
  - Hence it makes sense to have the attacker apply it

inductive-set
synth :: *msg set* => *msg set*
for *H* :: *msg set*
where
Inj [*intro*]: $X \in H \Longrightarrow X \in synth\ H$
| *Agent* [*intro*]: $Agent\ agt \in synth\ H$
| *Number* [*intro*]: $Number\ n \in synth\ H$
| *Hash* [*intro*]: $X \in synth\ H \Longrightarrow Hash\ X \in synth\ H$
| *MPair* [*intro*]: $[|X \in synth\ H;\ Y \in synth\ H|] \Longrightarrow \{|X, Y|\} \in synth\ H$
| *Crypt* [*intro*]: $[|X \in synth\ H;\ Key(K) \in H|] \Longrightarrow Crypt\ K\ X \in synth\ H$

Basic lemmas

- All of these can be proved
  - analz H included in parts H
  - analz(analz H) = analz H
  - synth(synth H) = synth H
  - analz(synth H) = analz H $\cup$ synth H
  - Nonce N $\in$ synth H $\Rightarrow$ Nonce N $\in$ H
  - Crypt (K) X $\in$ synth H $\Rightarrow$ Crypt (K) X $\in$ H $\vee$ (X $\in$ synth H $\wedge$ K $\in$ H)
  - synth(analz H) is the hardest to evaluate, skipped
  - …
- Each lemma seves as rewriting rule for subsequent ones

Example

- One crucial function still remains to be defined… which one?

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : sign(B, N_a)$

```
Nil:  "[] ∈ flp"

Fake: "⟦evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))⟧
       ⟹ Says Spy B X # evsf ∈ flp"

FLP1: "⟦evs1 ∈ flp; Nonce Na ∉ used evs1⟧
       ⟹ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "⟦evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2⟧
       ⟹ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "⟦evsr ∈ flp; Says A B X ∈ set evsr⟧
       ⟹ Gets B X # evsr ∈ flp"
```

Main operators: knows

- It implements knowledge of an agent over a given trace

consts

$bad :: agent\ set$

$knows :: agent => event\ list => msg\ set$

specification (bad)

$Spy\text{-}in\text{-}bad\ [iff]: Spy \in bad$

$Server\text{-}not\text{-}bad\ [iff]: Server \notin bad$

Main operators: knows

```
primrec
  knows-Nil:   knows A [] = initState A
  knows-Cons:
    knows A (ev # evs) =
      (if A = Spy then
       (case ev of
          Says A' B X => insert X (knows Spy evs)
        | Gets A' X => knows Spy evs
        | Notes A' X =>
            if A' ∈ bad then insert X (knows Spy evs) else knows Spy evs)
       else
       (case ev of
          Says A' B X =>
            if A'=A then insert X (knows A evs) else knows A evs
        | Gets A' X    =>
            if A'=A then insert X (knows A evs) else knows A evs
        | Notes A' X    =>
            if A'=A then insert X (knows A evs) else knows A evs))
```

Encoding the threat model

- The set of all messages that the attacker can synthesize from the analysis of her observation of the traffic on a given trace evsf is defined as synth(analz(knows Spy evsf))
- Hence the Fake rule, present in each protocol model, introduces an attacker who sends out any of the messages she can fake, as described above, to anyone
- Clearly, no bound stated on the attacker's capabilities, contrarily to other approaches, such as state enumeration
- This is perhaps the main strength of the Inductive Method
- We skip the symbolic analysis of synth(analz(knows Spy evsf)), which does have a set that bounds it, contrarily to intuition, handled automatically by the dedicated proof method spy_analz whose explanation is a separate course!

The example protocol, fully explained

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : sign(B, N_a)$

```
Nil:  "[] ∈ flp"

Fake: "⟦evsf ∈ flp; X ∈ synth(analz(knows Spy evsf))⟧
       ⟹ Says Spy B X # evsf ∈ flp"

FLP1: "⟦evs1 ∈ flp; Nonce Na ∉ used evs1⟧
       ⟹ Says A B {|Agent A, Nonce Na|} # evs1 ∈ flp"

FLP2: "⟦evs2 ∈ flp; Gets B {|Agent A, Nonce Na|} ∈ set evs2⟧
       ⟹ Says B A (Crypt (priSK B) (Nonce Na)) # evs2 ∈ flp"

Recp: "⟦evsr ∈ flp; Says A B X ∈ set evsr⟧
       ⟹ Gets B X # evsr ∈ flp"
```

An example trace of the example protocol

- Grey events must be pruned
  - Why?
- Precisely, this trace belongs to the model of the example protocol

1. $A \rightarrow B : A, N_a$
2. $B \rightarrow A : sign(B, N_a)$

```
Says Spy D (Nonce Nc)
Notes Spy (Nonce Nc)
Notes Spy (Nonce Nc)
Says B C (Crypt (priSK B)(Nonce Nc))
Gets B {|Agent C, Nonce Nc|}
Notes B (Nonce Nc)
Says Spy D (Nonce Na')
Says C B {|Agent C, Nonce Nc|}
Notes Spy (Nonce Na')
Says A B {|Agent A, Nonce Na'|}
Says A B {|Agent A, Nonce Na|}
```

Encoding the main protocol goals

- Secrecy of m encoded as: ∀ trace evs of the protocol model, it must be the case that m does not belong to what the attacker can analyse from her knowledge over evs
  - In practice evs is a free variable in the model, i.e. a generic trace of the model
  - I.e. evs ∈ flp. Key K ∉ analz(knows Spy evs)
- Authentication of A with B encoded as: ∃message m such that, ∀trace evs of the protocol model on which B receives m, it must have been A who sent m to B on the trace
  - In practice evs is a free variable in the model, i.e. a generic trace of the model
  - I.e. evs ∈ flp. If Gets B m appears on evs, then also Says A B m does
- Warning: notation still much simplified in this page!

Towards the proofs

- Safety properties: prove that in every event of every trace, the stated security goal holds
- More precisely, two forms of induction
  - Usual form for $\forall n \in Nat.\ P(n)$
    - Base case: P(0)
    - Induction step: $P(x) \Rightarrow P(x+1)$
    - Conclusion: $\forall n \in Nat.\ P(n)$
  - Minimal counterexample form
    - Conjecture: $\exists x.\ \neg P(x) \wedge \forall y<x.\ P(y)$
    - Prove contradiction
    - Conclusion: $\forall n \in Nat.\ P(n)$
- Typical proof scheme: build inductive formula, apply induction, simplify and, if needed, apply classical reasoner

Rules of thumb

- Build a conjecture to attempt to prove (to see if it is a theorem) by informally checking if each rule defining the protocol model preserves it
- After applying induction, be prepared to a number of subgoals in the proof state equal to the number of rules defining the protocol model
  - Hence the generic trace is the rule is conveniently numbered!
- Failing to prove a conjecture either
  - Means that you must concentrate more ☺
  - Or that you are facing a proof state that cannot be solved
    - It should be possible to understand why, what makes the conjecture fail
    - If your conjecture was a basic goal, then you probably found an attack!

An example theorem

- Observe theorem statement and theorem proof script
- Each “apply” applies a method – let’s explain the full syntax
- The proof script fails to show how the actual proof unfolding
  - It can be seen live!

```
theorem Spy-not-see-NA:
     [[Says A B (Crypt(pubEK B) {|Nonce NA, Agent A|}) ∈ set evs;
       A ∉ bad;  B ∉ bad;  evs ∈ ns-public]]
      ⟹ Nonce NA ∉ analz (spies evs)
apply (erule rev-mp, erule ns-public.induct)
apply simp-all
 apply (rule conjI, clarify)
apply clarify

apply (drule Fake-analz-insert [THEN subsetD], simp)
 apply simp
apply (blast dest: unique-NA intro: no-nonce-NS1-NS2)+
done
```

Another example: finding Lowe's attack

- Note
  - Protocol model name
  - "spies" abbreviates "knows Spy"
  - Object-level (single arrow) and meta-level (double arrow) implications

$$\textbf{lemma}\ [\![A \notin bad;\ \ B \notin bad;\ \ evs \in ns\text{-}public]\!]$$
$$\Longrightarrow Says\ B\ A\ (Crypt\ (pubEK\ A)\ \{\!|Nonce\ NA,\ Nonce\ NB|\!\}) \in set\ evs$$
$$\longrightarrow Nonce\ NB \notin analz\ (spies\ evs)$$

Object vs. meta level

- Object level is the level of the specification language
  - HOL in our case, which is also addressed as object logic
- Meta level is the level of the abstract reasoning
- Example 1 (introducing meta-level implication)
  - Suppose you know that $P(X) \vee Q(X)$
  - Then I also tell you that $\neg P(X)$
  - So you deduce that $Q(X)$
  - The entire reasoning is written $((P(X) \vee Q(X)) \wedge \neg P(X)) \Rightarrow Q(X)$
- Example 2 (using both implications)
  - Suppose you know that $P(X) \rightarrow Q(X)$
  - Then I also tell you that $P(X)$
  - So you deduce $Q(X)$
  - The entire reasoning is written $((P(X) \rightarrow Q(X)) \wedge P(X)) \Rightarrow Q(X)$

Finding Lowe's attack: proof script

- Cannot be understood without interactively inspecting how the proof state evolves…

apply (*erule ns-public.induct*, *simp-all*, *spy-analz*)

apply *blast*

apply (*blast intro*: *no-nonce-NS1-NS2*)

apply *clarify*
thm *unique-NB*
thm *Says-imp-knows-Spy* [*THEN parts.Inj*, *THEN unique-NB*]
apply (*frule-tac* $A' = A$ **in**
Says-imp-knows-Spy [*THEN parts.Inj*, *THEN unique-NB*], *auto*)
apply (*rename-tac* C B' *evs3*)

This is the attack!

1. $\bigwedge C\ B'\ evs3.$
$[\![A \notin bad;\ B \notin bad;\ C \in ns\text{-}public;$
$Says\ A\ B'\ (Crypt\ (pubK\ B')\ \{\!|Nonce\ NA,\ Agent\ A|\!\})\ \in\ set\ C;$
$Says\ evs3\ A\ (Crypt\ (pubK\ A)\ \{\!|Nonce\ NA,\ Nonce\ NB|\!\})$
$\in\ set\ C;$
$B' \in bad;$
$Says\ B\ A\ (Crypt\ (pubK\ A)\ \{\!|Nonce\ NA,\ Nonce\ NB|\!\})\ \in\ set\ C;$
$Nonce\ NB \notin analz\ (knows\ Spy\ C)]\!]$
$\Longrightarrow False$

oops

Demo!

- Interacting with Isabelle
  - Through Message.thy
  - Then Event.thy
  - Then Public.thy
  - And finally NS_Public_Bad.thy with its failed proof attempt of authentication of Alice with Bob, which indicates Lowe's attack

Formalising non-repudiation as a trace property

- Similar to authentication!
  - Must prove that if an agent gets/receives a message then the other agent gets/received a (the same or possibly another) message
  - Those generic messages contribute to forming the non-repudiation evidence, as seen
- E.g. if sender has NRR then receiver has NRO
- We expect a natural specification and analysis in the Inductive Method

Inductive protocol model of ZG

```
Nil:  "[] ∈ zg"

Fake: "⟦evsf ∈ zg;  X ∈ synth (analz (knows Spy evsf))⟧
       ⟹ Says Spy B X  # evsf ∈ zg"

Reception: "⟦evsr∈zg; Says A B X ∈ set evsr⟧ ⟹ Gets B X # evsr ∈ zg"

ZG1: "⟦evs1 ∈ zg;  Nonce L ∉ used evs1; C = Crypt K (Number m);
       K ∈ symKeys;
       NRO = Crypt (priK A) {|Number f_nro, Agent B, Nonce L, C|}⟧
   ⟹ Says A B {|Number f_nro, Agent B, Nonce L, C, NRO|} # evs1 ∈ zg"

ZG2: "⟦evs2 ∈ zg;
       Gets B {|Number f_nro, Agent B, Nonce L, C, NRO|} ∈ set evs2;
       NRO = Crypt (priK A) {|Number f_nro, Agent B, Nonce L, C|};
       NRR = Crypt (priK B) {|Number f_nrr, Agent A, Nonce L, C|}⟧
   ⟹ Says B A {|Number f_nrr, Agent A, Nonce L, NRR|} # evs2  ∈  zg"

ZG3: "⟦evs3 ∈ zg; C = Crypt K M; K ∈ symKeys;
       Says A B {|Number f_nro, Agent B, Nonce L, C, NRO|} ∈ set evs3;
       Gets A {|Number f_nrr, Agent A, Nonce L, NRR|} ∈ set evs3;
       NRR = Crypt (priK B) {|Number f_nrr, Agent A, Nonce L, C|};
       sub_K = Crypt (priK A) {|Number f_sub, Agent B, Nonce L, Key K|}⟧
   ⟹ Says A TTP {|Number f_sub, Agent B, Nonce L, Key K, sub_K|}
       # evs3 ∈ zg"

ZG4: "⟦evs4 ∈ zg; K ∈ symKeys;
       Gets TTP {|Number f_sub, Agent B, Nonce L, Key K, sub_K|} set evs4;
       sub_K = Crypt (priK A) {|Number f_sub, Agent B, Nonce L, Key K|};
       con_K = Crypt (priK TTP) {|Number f_con, Agent A, Agent B,
                                  Nonce L, Key K|}⟧
   ⟹ Says TTP Spy con_K
       # Notes TTP {|Number f_con, Agent A, Agent B, Nonce L, Key K, con_K|}
       # evs4 ∈ zg"
```

Handling the new threat model

- Rather than defining it from scratch, hacked existing one
- Remember set of bad agents, which include the attacker
- Let us define: broken = bad – {Spy}
- Assuming that an agent is not broken still allows him to be the attacker, as is appropriate for non-repudiation protocols

Proving validity of evidence

- Whenever con_K appears, it originated with TTP
  - Stronger in terms of used or in terms of parts?

```
lemma con_K_validity:
     "[[con_K ∈ used evs;
       con_K = Crypt (priK TTP)
                   {|Number f_con, Agent A, Agent B, Nonce L, Key K|};
       evs ∈ zg]]
  ⟹Notes TTP{|Number f_con, Agent A, Agent B, Nonce L, Key K, con_K|}
          ∈ set evs"
```

- Analogous lemma for sub_K

More on validity of evidence

```
theorem NRO_validity:
     "[[Gets B {Number f_nro, Agent B, Nonce L, C, NRO} ∈ set evs;
       NRO = Crypt (priK A) {Number f_nro, Agent B, Nonce L, C};
       A ∉ broken;  evs ∈ zg]]
    ⟹ Says A B {Number f_nro, Agent B, Nonce L, C, NRO} ∈ set evs"
```

```
theorem NRR_validity:
     "[[Gets A {Number f_nrr, Agent A, Nonce L, NRR} ∈ set evs;
       NRR = Crypt (priK B) {Number f_nrr, Agent A, Nonce L, C};
       B ∉ broken; evs ∈ zg]]
    ⟹ Says B A {Number f_nrr, Agent A, Nonce L, NRR} ∈ set evs"
```

Proving fairness

theorem *B_fairness_NRR:*

```
"⟦NRR ∈ used evs;
  NRR = Crypt (priK B) {Number f_nrr, Agent A, Nonce L, c};
  NRO = Crypt (priK A) {Number f_nro, Agent B, Nonce L, c};
  B ∉ bad; evs ∈ zg⟧
⟹ Gets B {Number f_nro, Agent B, Nonce L, c, NRO} ∈ set evs"
```

Fairness for *A* has a slightly different form: if *con_K* and *NRO* exist, then *A* holds *NRR*. We see how *con_K* gives fairness to *A*, who otherwise would be at a disadvantage because the first message gives evidence to *B*.

theorem *A_fairness_NRO:*

```
"⟦con_k ∈ used evs;
  NRO ∈ parts (knows Spy evs);
  con_k = Crypt (priK TTP)
                {Number f_con, Agent A, Agent B, Nonce L, Key k};
  NRO = Crypt (priK A) {Numberf_nro, Agent B, Nonce L, Crypt k m};
  NRR = Crypt (priK B) {Number f_nrr, Agent A, Nonce L, Cryptk m};
  A ∉ bad;  evs ∈ zg⟧
⟹ Gets A {Number f_nrr, Agent A, Nonce L, NRR} ∈ set evs"
```

A glimpse at the proofs for ZG

- Demo on repository file Zhou-Gollmann.thy